



Testiranje i kvalitet softvera

IV – Vrste testiranja – strukturno testiranje

**Prof. Dragan Janković
Katedra za Računarstvo**

Sadržaj

- Unit testiranje (testiranje unita, jedinično)
- Integraciono
- Testiranje sistema
- Validaciono testiranje



UNIT TESTING

- Testiranje Unit-a – se fokusira na najmanje elemente (celine, module) u softveru.
 - U kontekstu OO softvera odgovara mu testiranje klasa.
- Korišćenje white-box testiranja
 - Specifikacija - dostupna
 - Kod - dostupan
- Obično ga realizuju sami programeri – nakon implementacije unita

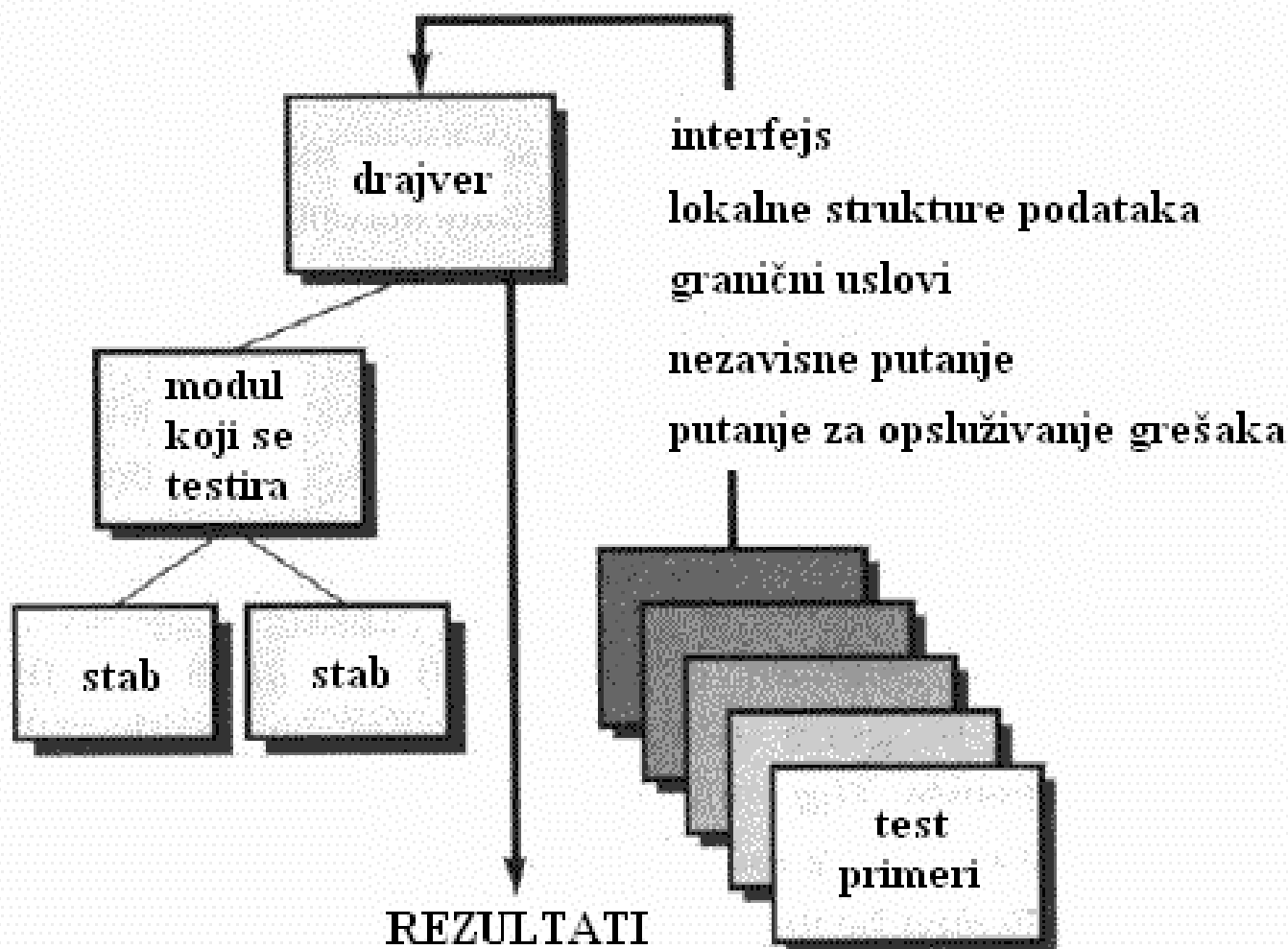


Okruženje za testiranje pojedinačnih modula

- Pošto komponenta nije nezavisan program, mora se dodatno razviti odgovarajuće softversko okruženje da bi se testirala nezavisno od drugih komponentata



Okruženje za testiranje pojedinačnih modula



Okruženje za testiranje...

- **Drajver** i/ili **stab** softver mora biti razvijen za svaki test pojedinačnog modula.
- U većini aplikacija **drajver** ne predstavlja ništa drugo, do glavni program koji prihvata ulazne podatke iz test primera, prosleđuje ih testiranom modulu, i potom prikazuje relevantne rezultate testa.
- **Stabovi** služe da zamene module koje poziva modul koji se testira (**stabovi** – zamena za pozivane module).
 - **Stab** ili “lažni potprogram” koristi interfejse pozivanih modula, odrađuje minimalnu obradu podataka, ispisuje potvrdu ulaza i na kraju vraća kontrolu modulu koji se testira.



UNIT TESTING – faktori greške

Pri generisanju Unit testova moraju se uzeti u razmatranje mnogi faktori koji mogu da dovedu do greške:

- ☐ Interfejs
- ☐ Lokalne strukture podataka
- ☐ Uslovi
- ☐ Nezavisni putevi
- ☐ Putevi za obradu grešaka
- ☐ itd.



Generisanje Unit Testa

■ Razmatranje interfejsa

- ☐ # ulaznih parametara = # argumenata?
- ☐ Atributi parametara i argumenata se poklapaju?
- ☐ Da li je redosled korektan (ako je to važno)?
- ☐ Broj i redosled argumenata korektan?
- ☐ Reference na parametre nisu pridružene ulazima?
- ☐ Pokušaj modifikacije čisto ulaznih parametara?
- ☐ Definicije globalnih parametara konzistentne?
- ☐ Ograničenja su uneta kao argumenti?
- ☐ Podrazumevani parametri
- ☐ Implicitne konverzije tipova (ugrađeni, korisnički(klase))



Generisanje Unit Testa

■ Razmatranje eksternih I/O

- ☐ Atributi fajlova korektni?
- ☐ OPEN/CLOSE korektni?
Format specifikacije odgovaraju I/O naredbama?
- ☐ Buffer size odgovara record size?
- ☐ Fajlovi su otvoreni pre korišćenja?
- ☐ EOF se koristi korektno?
- ☐ I/O greške se hendluju?
- ☐ Tekstualne greške u ulazima?



Generisanje Unit Testa

■ Razmatranje struktura podataka

- ☐ Neodgovarajuće ili nekonzistentno postavljanje tipova?
- ☐ Pogrešna inicijalizacija ili default vrednosti?
- ☐ Nekorektna imena promenljivih? (jednine, množine)
- ☐ Nekonzistentni tipovi podataka?
- ☐ Podkoračenje, prekoračenje i greške adresiranja?
- ☐ Pointeri
- ☐ „Pristup“ podacima van opsega strukture



Generisanje Unit Testa

- Kriterijum – npr. Testovi mora da pokriju sve puteve
- Mora da se ispitaju greške računanja:
 - ☐ Nekorektna aritmetika
 - ☐ Nekorektna inicijalizacija
 - ☐ Netačna preciznost
 - ☐ Nekorektna simbolička reprezentacija izraza
- Ostala potrebna testiranja
 - ☐ Nekompatibilni tipovi podataka u poredjenjima
 - ☐ Nekorektni logički operatori ili prioritet
 - ☐ Problemi poredjenja (npr. == nad podacima tipa float)
 - ☐ Problemi sa petljama (granice petlji, uslov za kraj, modifikacija indeksa petlje)



Generisanje Unit Testa

■ Testovi upravljanja greškama

- ☐ Exception-handling (EH) je nekorektan?
- ☐ Opis grešaka je nerazumljiv, nedovoljan ili nekorektan?
- ☐ Greške dovode do pada sistema pre nego što je kompletiran error handling?
- ☐ Hijerarhija grešaka (nasleđivanje tipova grešaka kod korisničkih klasa)

■ OO kod, Web programi, ...

■ Višeslojne aplikacije (komunikacija između slojeva sa EH)



Integraciono testiranje

- Da li je dovoljno samo unit testiranje?
- Svrha integracionog testiranja?
- Ukoliko su svi moduli uspešno prošli pojedinačno testiranje, zašto bismo imali bilo kakve nedoumice da li će ispravno raditi kada se povežu zajedno u celinu?



Svrha integracionog testiranja

- Problem je, naravno, u njihovom međusobnom povezivanju.
 - Dok prolaze kroz interfejs, podaci se mogu izgubiti; jedan modul može imati nepovoljan uticaj na neki drugi modul;
 - određena kombinacija podfunkcija može dati neželjen rezultat glavne funkcije;
 - nepreciznosti koje su prihvatljive u pojedinačnim modulima, mogu u međusobnom povezivanju da narastu do neprihvatljivo velikih vrednosti;
 - globalne strukture podataka mogu da predstavljaju problem (više modula im pristupa i menja ih)
 - itd.



Integraciono testiranje

- Sistematski pristup u konstruisanju programske strukture pri čemu se testovima otkrivaju **greške vezane za interfejse**.
- Cilj: Otkriti greške koje se odnose na kompatibilnost modula i način njihovog povezivanja
- Po V-modelu glavni nivoi granularnosti kod testiranja su testiranje: modula, integracija, sistema i prihvatno testiranje.



Integraciono testiranje

- Integracionim testiranjem se ne otklanjaju problemi u modulima.
- Kvalitet sistema zavisi od kvaliteta delova.
- Često nekritični moduli mogu imati značajan uticaj i efekat.
- **Primer:** 2004 god. Buffer overflow u široko korišćenoj biblioteci za čitanje PNG (portable network graphics) izazvala niz problema u web browserima i mail klijentima na Windowsu, Linuxu i Mac OS X



Nekompatibilnost modula

- Ariane 5 incident (4.7.1996. pad)
- Uzrok: nekompatibilnost modula sa zahtevima za Ariane 5
- Modul je za Ariane 4 koja je nešto manja od Ariane 5 i imala je manju horizontalnu brzinu
- Nije testiran za trajektoriju Ariane 5 i uslove njenog leta



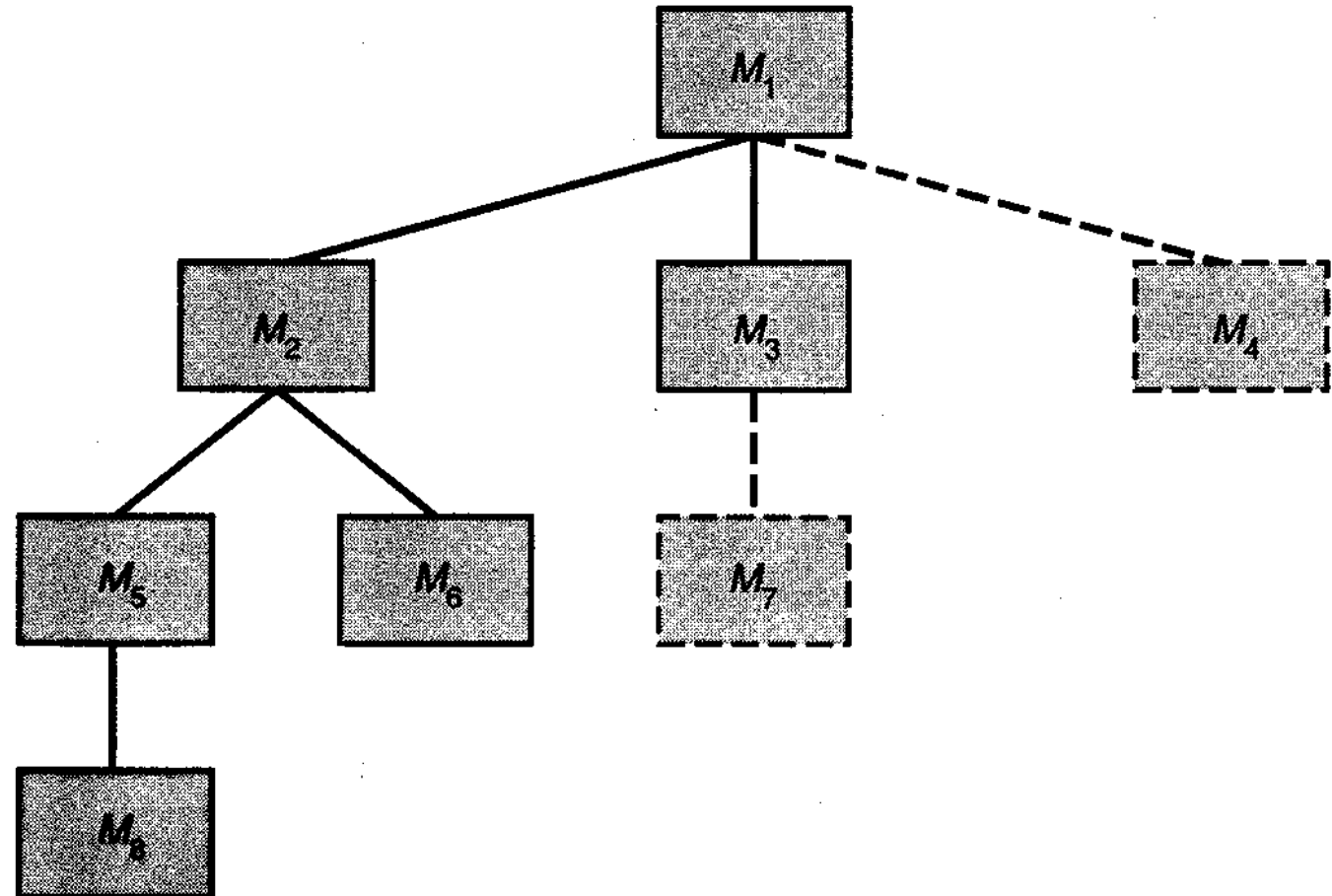
Vrste integracionog testiranja

- Postoji tendencija ka *Ne-Inkrementalnoj integraciji* tj. integracija po principu “velikog praska”. Sve komponente (moduli) se unapred iskombinuju i onda program testira kao celina - uglavnom rezultuje u velikim greškama.
- Inkrementalna integracija - program se konstruiše i testira u malim segmentima.
 - ☐ Top-Down Integraciono testiranje
 - ☐ Bottom-Up Integraciono testiranje
- Fokus: Testiranje Integracije kritičnih modula



INTEGRACIONO TESTIRANJE

Integracija od vrha ka dnu (Top-down)



Top – down integracija

- Početak konstruisanja i testiranja od glavnog modula (glavnog programa).
 - Stub-ovi menjaju sve podmodule.
- Stub-ovi se menjaju aktuelnim modulima jedan po jedan.
- Testovi se dodaju kad se integriše svaki novi modul.
- Po završetku celokupnog skupa testova, sledeći stub se menja realnim modulom.
- Regresiono testiranje može da se izvrši da se proverí da nisu možda napravljene nove greške.



Integracije od vrha ka dnu

- Pri integraciji se kreće na dole niz kontrolnu hijerarhiju, počevši od glavnog kontrolnog modula (glavnog programa).
- Moduli podređeni glavnom se uključuju u strukturu po jednoj od strategija:
 - ☐ “po dubini” ili
 - ☐ “po širini”.

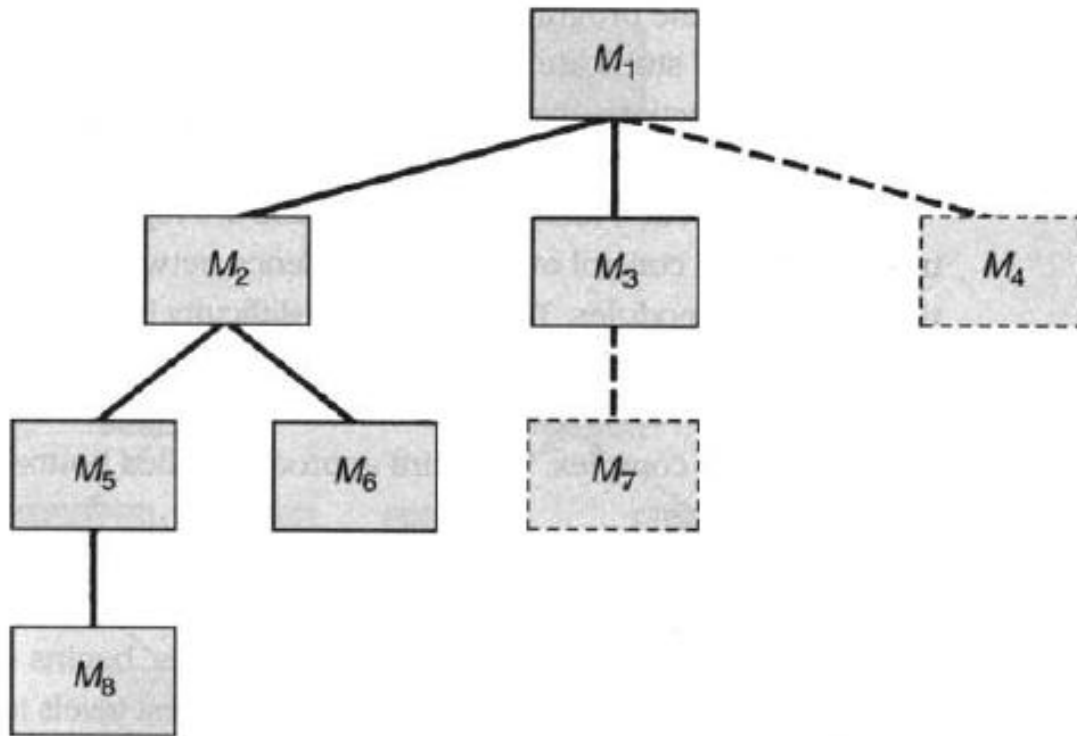


Integracije od vrha ka dnu

- Strategija “**po dubini**” će izvršiti integraciju svih komponenti na glavnoj kontrolnoj putanji strukture programa. Izbor glavne putanje je donekle proizvoljan i zavisi od karakteristika specifikacije programa (aplikacije).
- Strategija integracije “**po širini**” uključuje sve neposredno podređene komponente na jednom nivou krećući se kroz strukturu programa horizontalno.



Primer

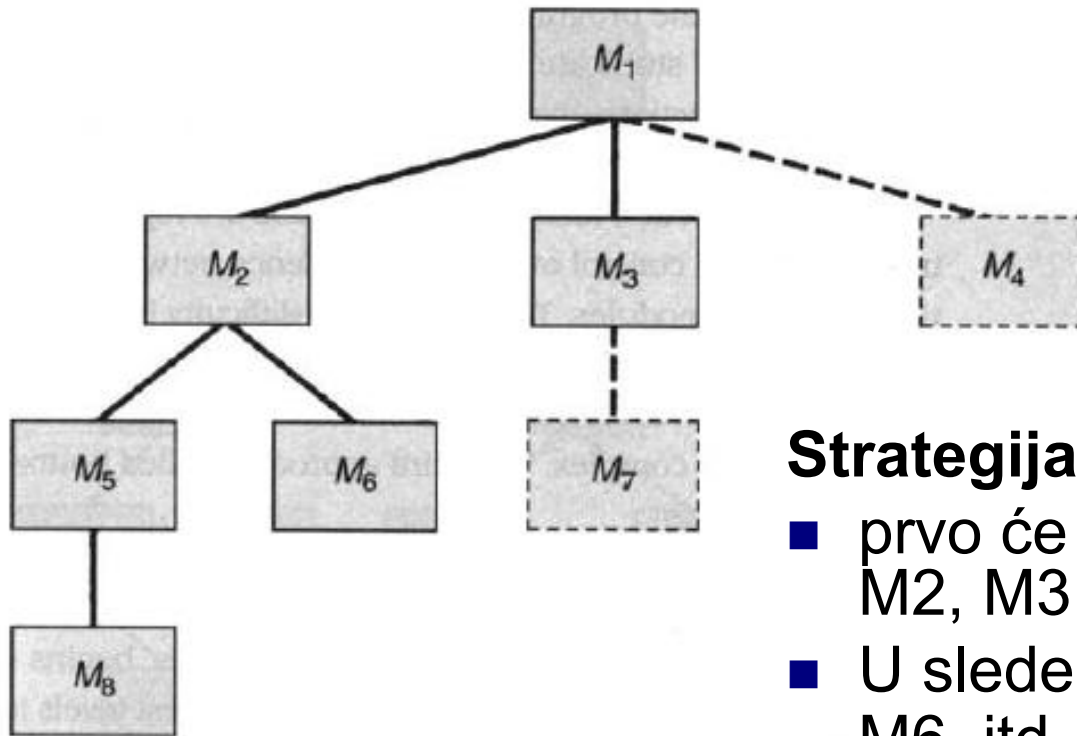


Strategija “po dubini”:

- Izborom leve putanje, prvo će biti integrisane komponente M1, M2, M5.
- Zatim će biti integrisani M8, ili (radi ispravnog funkcionisanje M2) M6.
- Onda će biti napravljene i integrisane, srednja, a zatim i desna kontrolna putanja.



Primer



Strategija “po širini”:

- prvo će se integrisati komponente M2, M3 i M4.
- U sledećem kontrolnom nivou M5, M6, itd.



Integracija od vrha ka dnu

- Integraciona strategija “od vrha ka dnu” verifikuje ranije, u procesu testiranja, glavnu kontrolu ili tačke odlučivanja.
- U dobro izdeljenoj strukturi programa, odluke se donose na višim hijerarhiskim nivoima i zbog toga se prvo tu sa njima i susrećemo. Ako imamo probleme u glavnoj kontrolnoj strukturi, njihovo rano otkrivanje je vrlo bitno.
- Ukoliko odaberemo integraciju “po dubini”, možemo implementirati i demonstrirati upotrebu softvera u potpunosti (deo funkcionalnosti).
- U ranim fazama razvoja demonstracije funkcionalnih mogućnosti programa grade poverenje, kako kod onoga koji razvija taj program, tako i kod naručioca.
- Kod strategije “od vrha ka dnu” u praksi se mogu javiti neki logistički problemi.



Integracija od vrha ka dnu

- Jedan od najčešćih problema javlja se kada se zahteva obrada u nižim hijerarhiskim nivoima, kako bi se adekvatno testirali viši nivoi.
- Na početku testiranja “od vrha ka dnu”, stabovi zamenjuju module na nižim hijerarhiskim nivoima. Zbog toga nikakvi značajni podaci ne mogu da cirkulišu od nižih ka višim strukturama programa.
- Zbog ovoga ostavljena su tri moguća izbora:
 1. Da se odlože mnogi testovi, dok se stabovi ne zamene stvarnim modulima;
 2. Da se konstruišu složeniji stabovi koji će vršiti ograničene obrade i tako simulirati stvarne module;
 3. Ili da se integracija strukture programa vrši od dna ka vrhu (naviše).



Posledice odluka

- **Prvi pristup** (odlaganje nekih testova) uzrokuje da izgubimo kontrolu nad vezom između određenih testova i ubacivanja (ugradnje) određenih modula. To može voditi ka teškoćama u određivanju uzroka greškama, što za posledicu ima narušavanje vrlo stroge prirode strategije “od vrha ka dnu”.
- **Drugi pristup** (složeniji stabovi) može da vrši posao, ali neizostavno vodi ka povećanju cene i dodatnom vremenu i poslu projektovanja sve složenijih i složenijih stabova.
- **Treći pristup**, koji je nazvan integracijom “od dna ka vrhu” biće razmotren kasnije.



Karakteristike Top-Down integracionog testiranja

■ Prednosti:

- Rano verifikuje glavne kontrolne tačke ili tačke odluke u testiranju.
- Korišćenjem integracionog testiranja po dubini, kompletna funkcija softvera može da se demonstrira.

■ Nedostaci:

- Pošto stub-ovi menjaju module na nižim nivoima, ne mogu značajni podaci da budu prosleđeni u glavni modul.

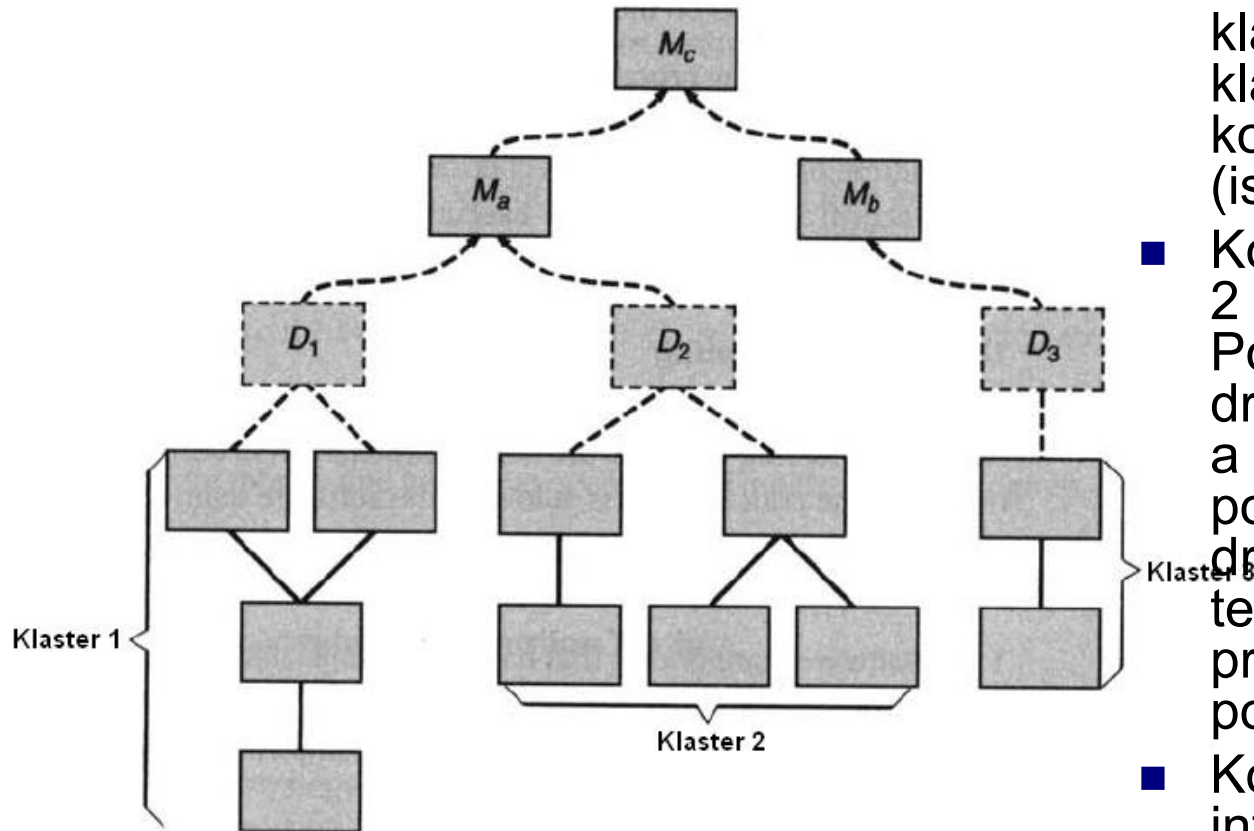


Bottom up pristup

- U ovom pristupu konstruisanje i testiranje počinje od modula na najnižim nivoima.
 - Moduli na nižim nivoima se kombinuju u **klaster**.
 - Pišu se **drajveri** (kontrolni program za testiranje) da bi mogli da se koordiniraju ulazi i izlazi testova.
 - Klaster se testira.
 - Drajveri se uklanjaju i klasteri se kombinuju uz pomeranje naviše po hijerarhiji.
 - Pošto se komponente integrišu od dna ka vrhu, obrada podataka koja se vršila u podređenim nivoima hijerarhije je sada dostupna (jer su i podređeni moduli već integrisani), pa su zbog toga i stabovi suvišni.



Primer



- Komponente se grupišu u klaster 1, 2 i 3. Svaki od klastera se, ponaosob, testira korišćenjem drajvera (isprekidani blok).
- Komponente u klasterima 1 i 2 su podređene modulu M_a . Po završetku testiranja drajveri D_1 i D_2 se uklanjaju, a klasteri se direktno povezuju na M_a . Slično se i drajver D_3 , koji je služio za testiranje klastera 3, uklanja pre nego što se ceo klaster 3 poveže na modul M_b .
- Konačno će i M_a i M_b biti integrisani sa M_c itd.



Karakteristike Bottom-Up integracionog testiranja

Bottom Up pristup

☐ Prednosti:

- Lakše generisanje testova i nema stubova.

☐ Nedostaci:

- Program kao celina ne postoji sve dok se ne integriše i poslednji modul.

Sendvič testiranje:- kombinovani pristup

- ☐ Top-down pristup za gornje nivoe i Bottom-up pristup za podnivoe.



Komentari o integracionom testiranju

- Kako teče integraciono testiranje, onaj ko testira (tester) trebalo bi da uoči *kritične module*.
- **Kritični modul** ima jednu ili više od sledećih nabrojanih karakteristika:
 1. Obrađuje nekoliko softverskih zahteva;
 2. Ima visok nivo kontrole (obično se nalazi na višim nivoima strukture programa);
 3. Složen je ili je verovatnoća pojavljivanja grešaka kod njega veoma velika (ciklomatska (cyclomatic) složenost može biti indikator);
 4. Ima tačno određene zahteve u pogledu performansi koje treba da zadovolji.
- Kritične module treba testirati što je moguće ranije.
- Dodatno, regresiono testiranje treba fokusirati na testiranje kritičnih modula.



Voditi računa i o

- Jezici koji podržavaju dinamičko povezivanje tj. polimorfizam jer problemi mogu nastati u toku integracije
- Memory leaks
- Nekompatibilnost jedinica
 - Primer: metri i jardi (uzrok za gubljenje Mars Climate Satelita septembra 1999.)
- tmp folderi (dva modula koriste tmp lociran na istom mestu)

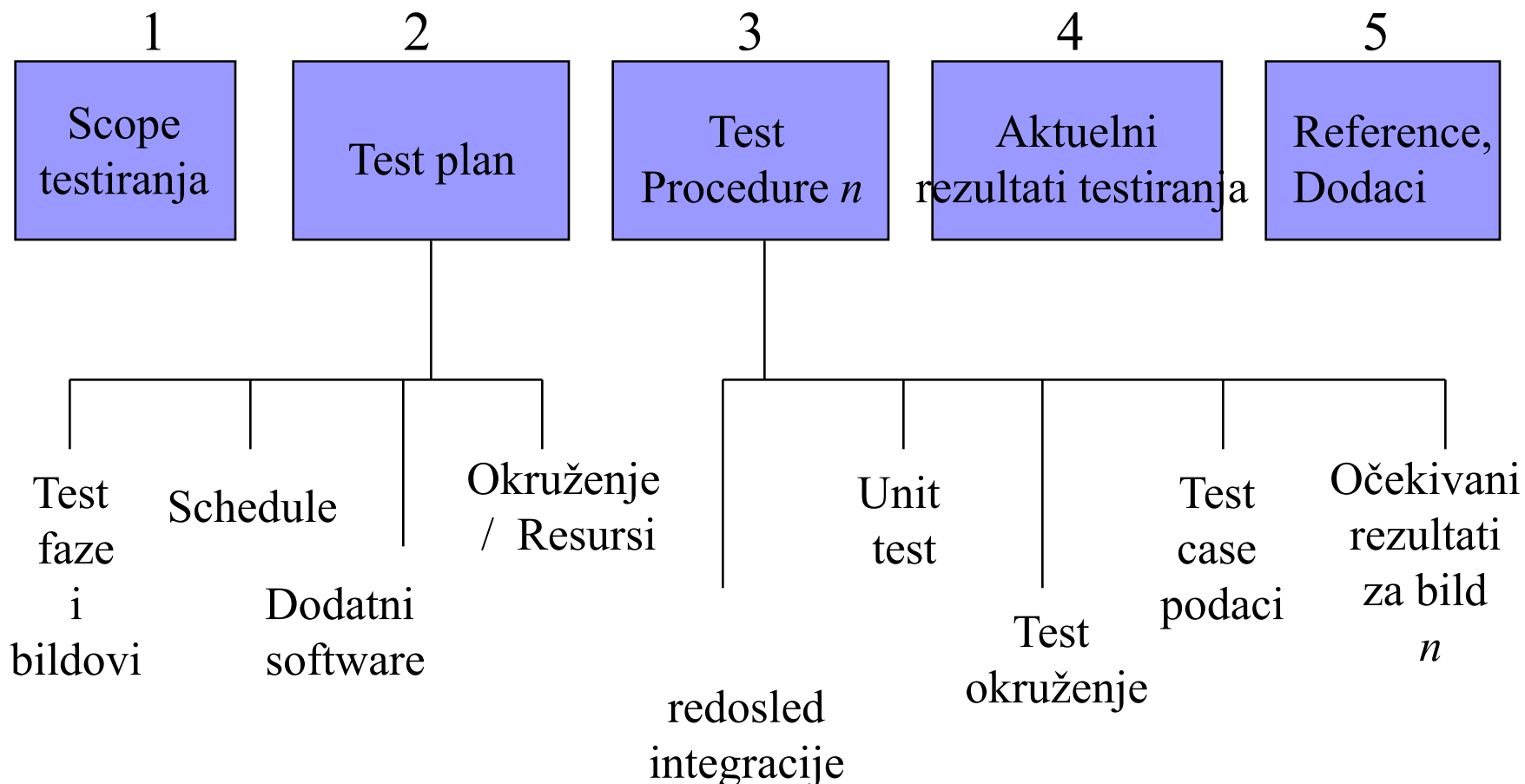


Greške

- Nekonzistentna interpretacija parametara i vrednosti (nekompatibilni npr. metri vs jardi)
- Neodgovarajući opseg vrednosti ili kapacitet ili veličina ograničenja (Apache 2 Web server između 2.0.35 i 2.0.5 buffer overflow)
- Bočni efekti na parametrima i resursima
- Nepostojeća funkcionalnost ili pogrešno shvaćena funkcionalnost
- Nefunkcionalni problemi



Dokumentacija Integracionog Testiranja



Regresiono (regresivno) testiranje

- Uspešno testiranje (bilo koje vrste) rezultira otkrivanjem grešaka, a greške moraju biti otklonjene. Uvek kada se ispravljaju greške u softveru, konfiguracija softvera se u nekom smislu menja (ili sam program, ili njegova dokumentacija, ili pripadajući podaci).
- **Regresiono testiranje** predstavlja aktivnost koja pomaže da se osigura da promene (zbog testiranja ili iz drugih razloga) ne izazovu neko neželjeno ponašanje ili nove greške.



Regresiono testiranje

- Regresiono testiranje se može sprovesti ručno, ponovnim izvršavanjem podskupa svih test primera, ili korišćenjem automatskih *capture-playback* alata. Ovi alati omogućavaju da se snime test primeri i njihovi rezultati, kako bi se naknadno opet sproveli i uporedili dobijeni rezultati



Regresiono testiranje

- Regresiono testiranje treba da sadrži tri različite klase testova :
 - Reprezentativni skup testova koji izvršavaju sve funkcije softvera
 - Dodatni testovi koji se fokusiraju na funkcije na koje učinjene promene mogu da utiču.
 - Testovi koji se fokusiraju na komponente koje se promenjene.



TESTIRANJE SISTEMA

- Niz testova kojima se verifikuje da su svi elementi sistema ispravno integrisani.
- Testiranja „višeg reda“:
 - ☐ Recovery testiranje
 - ☐ Testiranje sigurnosti
 - ☐ Stress testiranje
 - ☐ Testiranje performansi
 - ☐ Testiranje količine podataka
 - ☐ Testiranje upotrebljivosti



TESTIRANJE SISTEMA

- Testiranja „višeg reda“ (nastavak):
 - ☐ Testiranje integriteta
 - ☐ Uporedno testiranje
 - ☐ Testiranje zagušenja
 - ☐ Testiranje opterećenja
 - ☐ Testiranje konfiguracije
 - ☐ Testiranje instalacije



TESTIRANJE SISTEMA (1/5)

■ Recovery Testiranje:

- Softver se dovodi u stanje greške na različite načine i zatim se testira da li se oporavljanje (recovery) obavlja korektno.

■ Testiranje sigurnosti (Security Testing):

- Verifikacija zaštitnih mehanizama softvera.
- Da li su posmatrane funkcije dostupne onim i samo onim korisnicima kojima su namenjene



TESTIRANJE SISTEMA (2/5)

■ Stress testiranje:

- Izvršava se sistem na način gde se koristi abnormalna količina resursa, frekvenca ili veličina (ekstremna opterećenja, nedovoljno memorije ili drugih resursa, servisi koji nisu raspoloživi itd.).

■ Testiranje performansi:

- Testiranje runtime performansi sistema u kontekstu integrisanog sistema.



TESTIRANJE SISTEMA (3/5)

- Testiranje količine podataka (*volume testing*) – verifikovanje da li softver može obraditi veliku količinu podataka
- Testiranje upotrebljivosti (*usability testing*) – ocenjuju se estetski aspekti, konzistencija korisničkog interfejsa, korisnička dokumentacija i trenažni materijal.
- Testiranje integriteta (*integrity testing*) – ocenjuje se robusnost (otpornost na otkaze), konzistentna upotreba resursa i slično.
- Uporedni test (*benchmark testing*) – vrsta testa performansi koja upoređuje novi nepoznati sistem sa poznatim referentnim sistemom



TESTIRANJE SISTEMA (4/5)

- Test zagušenja (*contention testing*) – proverava da li softver može da zadovolji višestruke zahteve različitih aktera za istim resursom
- Test opterećenja (*load testing*) – vrsta testa performansi, kojim se procenjuju operativni limiti nepromenljivog sistema pod različitim opterećenjima, ili različitim konfiguracija sistema pri istom opterećenju. Merene veličine su najčešće protok i vreme odziva (srednja i vršna vrednost).



TESTIRANJE SISTEMA (5/5)

- Test konfiguracije (*configuration testing*) – testira ponašanje softvera u različitim hw/sw okruženjima
- Test instalacije (*installation testing*) – testira instaliranje softvera na različitim sistemima i u različitim situacijama (npr. prekid napajanja ili nedovoljno prostora na disku). Update – na koji način (redosled u složenim sistemima kao što su industrijski sistemi)?



VALIDACIONO TESTIRANJE

- Konačna provera da li softver ispunjava sve funkcionalne zahteve kao i performanse.
 - Isključivo korišćenje Black-box testiranja.
- Nakon svakog validacionog testa ili se potvrđuje da je softver po specifikaciji ili se pak detektuje neko odstupanje.
- Alpha i Beta testiranje
 - Alpha test – na strani developera od strane tima za testiranje u okruženju gde se vrši razvoj.
 - Beta test – na strani korisnika u živom okruženju od strane korisnika.



SIZE OF SAMPLE—6,877,000 STATEMENTS (COMMENTS INCLUDED)
 TOTAL REPORTED BUGS—16,209—BUGS PER 1000 STATEMENTS—2.36

1xxx REQUIREMENTS	1317	8.1%
11xx Requirements Incorrect	649	4.0%
12xx Requirements Logic	153	0.9%
13xx Requirements, Completeness	224	1.4%
15xx Presentation, Documentation	13	0.1%
16xx Requirements Changes	278	1.7%
	2624	16.2%
2xxx FEATURES AND FUNCTIONALITY		
21xx Feature/Function Correctness	456	2.8%
22xx Feature Completeness	231	1.4%
23xx Functional Case Completeness	193	1.2%
24xx Domain Bugs	778	4.8%
25xx User Messages and Diagnostics	857	5.3%
26xx Exception Condition Mishandled	79	0.5%
29xx Other Functional Bugs	30	0.2%
3xxx STRUCTURAL BUGS	4082	25.2%
31xx Control Flow and Sequencing	2078	12.8%
32xx Processing	2004	12.4%



4xxx DATA	3638	22.4%
41xx Data Definition and Structure	1805	11.1%
42xx Data Access and Handling	1831	11.3%
49xx Other Data Problems	2	0.0%
5xxx IMPLEMENTATION AND CODING	1601	9.9%
51xx Coding and Typographical	322	2.0%
52xx Style and Standards Violations	318	2.0%
53xx Documentation	960	5.9%
59xx Other Implementation	1	0.0%
6xxx INTEGRATION	1455	9.0%
61xx Internal Interfaces	859	5.3%
62xx External Interfaces, Timing, Throughput	518	3.2%
69xx Other Integration	78	0.5%
7xxx SYSTEM, SOFTWARE ARCHITECTURE	282	1.7%
71xx O/S Call and Use	47	0.3%
72xx Software Architecture	139	0.9%
73xx Recovery and Accountability	4	0.0%
74xx Performance	64	0.4%
75xx Incorrect Diagnostics, Exceptions	16	0.1%
76xx Partitions, Overlays	3	0.0%
77xx Sysgen, Environment	9	0.1%
8xxx TEST DEFINITION AND EXECUTION	447	2.8%
81xx Test Design Bugs	11	0.1%
82xx Test Execution Bugs	355	2.2%
83xx Test Documentation	11	0.1%
84xx Test Case Completeness	64	0.4%
89xx Other Testing Bugs	6	0.0%
9xxx OTHER, UNSPECIFIED	763	4.7%



Pitanja?

■ ?

■ ?

■ ?

